HDOC Day-59

****

Write a C program that reads a string s. Pass the string to a function named longest_unique_substring() using its address. Use pointers to pass the string between the calling function and longest_unique_substring(). The function longest_unique_substring() must:

- Find the longest substring of s that contains no repeated characters.
- Calculate and display the length of the longest substring.
- Treat uppercase and lowercase letters as different characters.
- Display 0 if the input string is empty.

1. Problem Understanding

The program reads a string and passes its address to longest_unique_substring(). The function checks contiguous substrings and finds the maximum length that contains no repeated characters. Character comparison is case-sensitive, so uppercase and lowercase letters are treated as different characters. If the string is empty, the result is 0.

2. Standard Test Cases

Test Case	String Input	Expected Output
1	abcabcbb	Length of longest substring: 3
2	bbbbb	Length of longest substring: 1
3	pwwkew	Length of longest substring: 3
4	aAbBcC	Length of longest substring: 6
5	Empty string	Length of longest substring: 0

3. Algorithm

Main Algorithm

Start.

Read a string from the user.

Remove the newline character, if present.

Pass the address of the string to longest_unique_substring().

Display the returned maximum length.

Stop.

Function: longest_unique_substring()

If the first character is the null character, return 0.

Initialize maxLength to 0.

For every starting position i, begin a new substring.

Move j from i toward the end of the string.

Before including s[j], check whether it already occurs between positions i and j - 1.

If a repeated character is found, stop extending that substring.

Otherwise calculate the current substring length and update maxLength if needed.

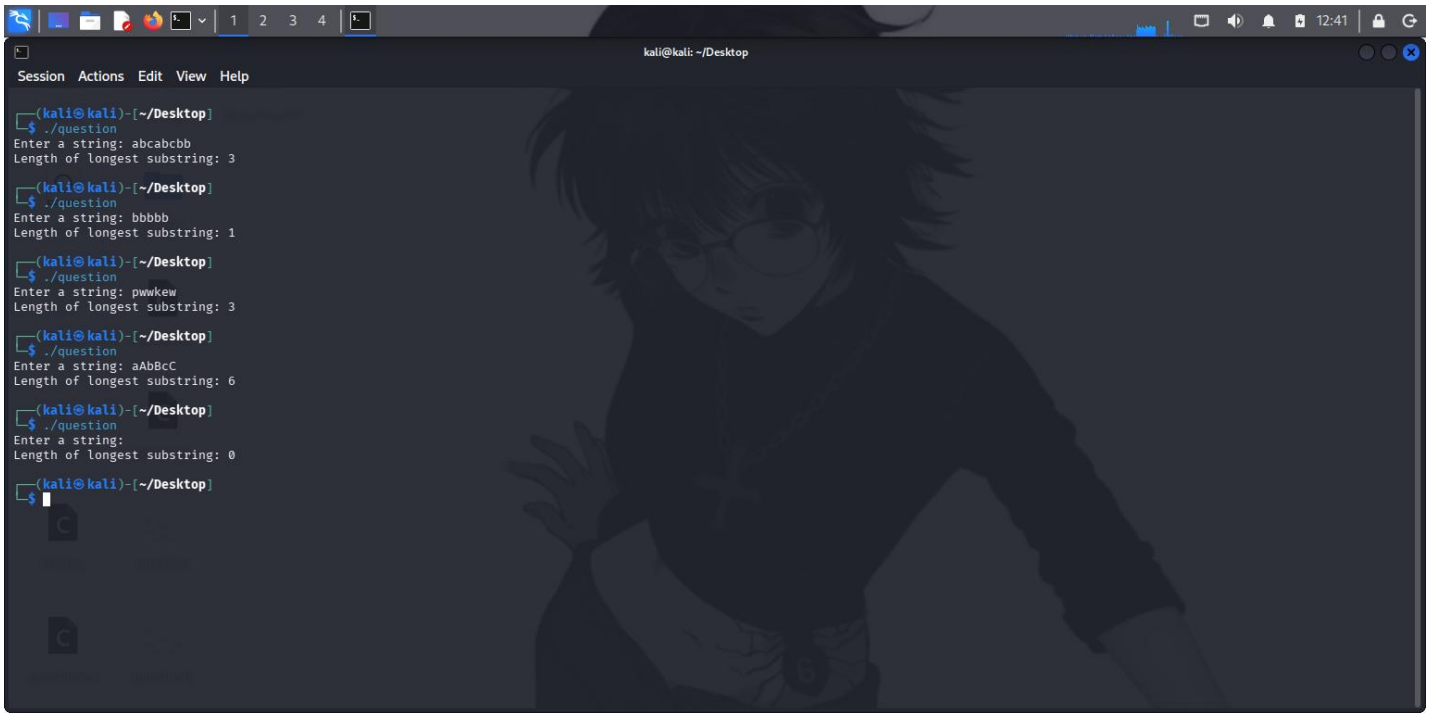
Return maxLength.

4. C Program

```
#include <stdio.h>
int longest_unique_substring(char *s)
{
   int i, j, k, maxLength = 0, repeated;
   if(*s == '\0')
      return 0;
   for(i = 0; *(s + i) != '\0'; i++)
   {
      for(j = i; *(s + j) != '\0'; j++)
      {
         repeated = 0;
         for(k = i; k < j; k++)
         {
            if(*(s + k) == *(s + j))
            {
               repeated = 1;
               break;
            }
         }
         if(repeated == 1)
            break;
         if(j - i + 1 > maxLength)
            maxLength = j - i + 1;
      }
   }
   return maxLength;
}
int main()
{
   char s[100];
   int i = 0, result;
   printf("Enter a string: ");
   fgets(s, 100, stdin);
   while(s[i] != '\0')
   {
      if(s[i] == '\n')
      {
         s[i] = '\0';
         break;
      }
      i++;
```

```
    }
    result = longest_unique_substring(s);
    printf("Length of longest substring: %d\n", result);
    return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	String Input	Actual Result	Status
1	abcabcbb	3	Pass
2	bbbbb	1	Pass
3	pwwkew	3	Pass
4	aAbBcC	6	Pass
5	Empty string	0	Pass